Advanced Python Day 4

Real System Design using OOP (Bank System)

Goal: Understand how classes model real-world systems.

Real-Life Example: Bank System

In a real bank context:

- → Many different customers exist within the same system.
- → Each customer has a specific Name and Account Balance.
- → Customers can independently deposit money.
- → Customers can withdraw money based on rules.
- → Customers can check their current balance anytime.

> **Are customers different? YES.**
> **Do they follow the exact same structural template? YES. This is why classes exist.**
>
> — Fundamental OOP Concept

Real Life → Python Mapping

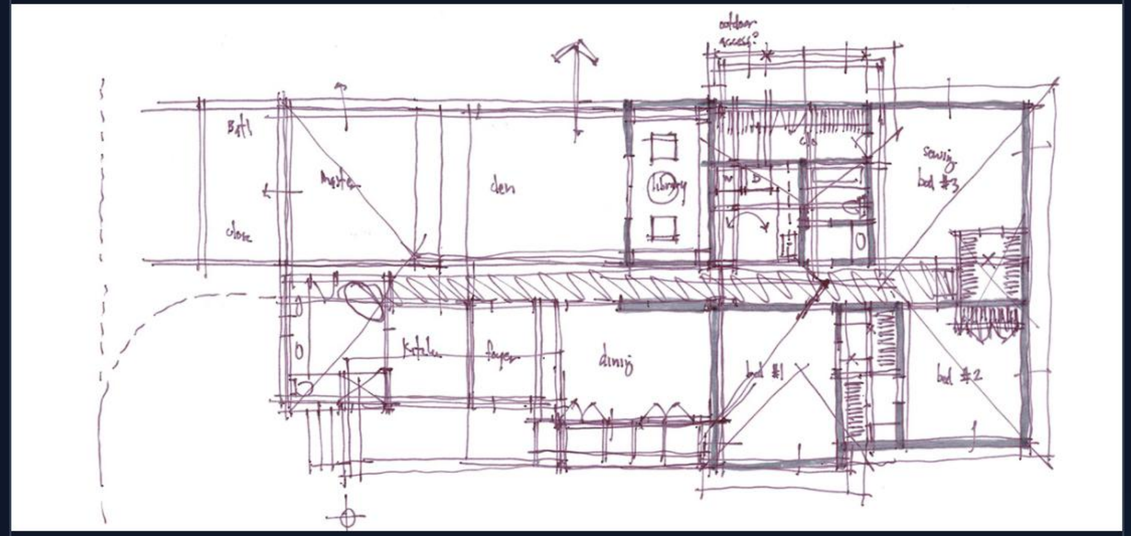
Real Life Concept	Python OOP Concept
Bank account type/structure	**Class** (The Blueprint)
Individual Customer account	**Object** (The Instance)
Account Balance, Name	**Attribute** (Data / Variables)
Deposit, Withdraw Actions	**Methods** (Behavior / Functions)

Step 1 — Create the Blueprint

We start by defining the class. Think of this as the architectural blueprint for every bank account that will ever exist in our system.

```
class BankAccount:
    pass
```

Currently, the blueprint is created, but there are no details or behaviors attached yet.



Step 2 & 3 — Initialization & Objects

Add Details (Constructor)

```
class BankAccount:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
```

The `__init__` method automatically runs when an account is made. Every account needs a starting Name and Balance.

Create Customers (Objects)

```
acc1 = BankAccount("Saurav", 1000)
acc2 = BankAccount("Rahul", 2000)
```

We just created two distinct objects from our blueprint. Each account operates completely independently.

Step 4 & 5 — Adding Behaviors

Check Balance Method

```
def show_balance(self):
    print(self.name, "balance:", self.balance)

# Usage:
acc1.show_balance()
```

Allows an object to access and display its current internal state securely.

Deposit Money Method

```
def deposit(self, amount):
    self.balance += amount
    print(amount, "deposited")
```

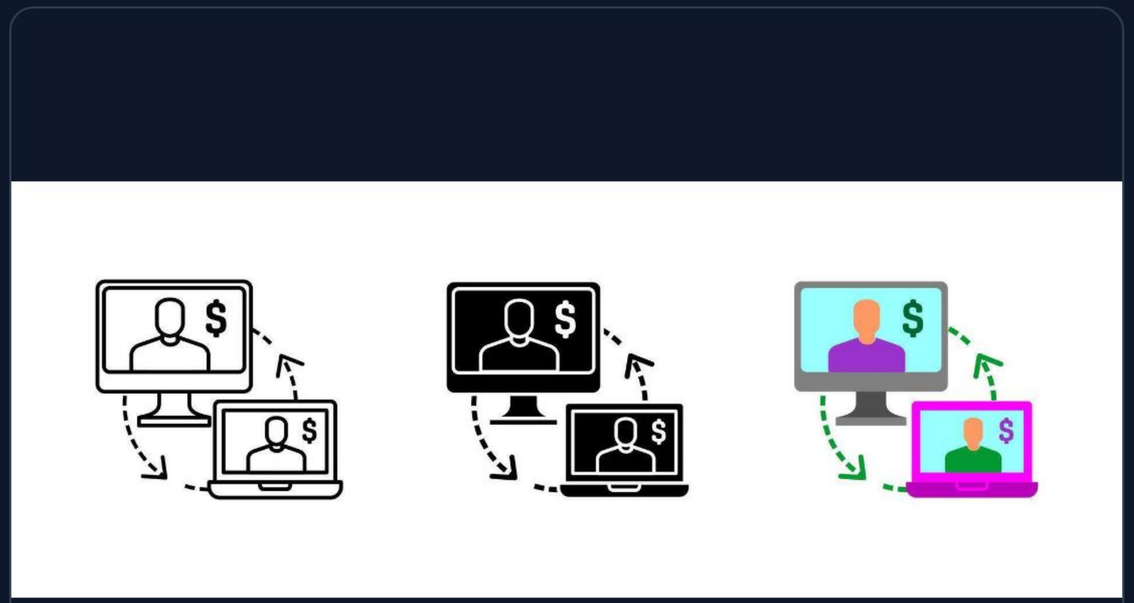
Modifies the state. The specific object's balance increases automatically upon execution.

Step 6 — Withdraw Logic

Implementing Real Rules

```
def withdraw(self, amount):
    if amount ⩽ self.balance:
        self.balance -= amount
        print("Withdraw successful")
    else:
        print("Insufficient balance")
```

This mimics a real-world constraint. A customer cannot withdraw more money than their object's specific balance allows.



Complete Bank Program

```python
class BankAccount:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def show_balance(self):
        print(self.name, "balance:", self.balance)

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
        else:
            print("Insufficient")
```

System Usage

```python
# 1. System provisions account
acc1 = BankAccount("Saurav", 1000)

# 2. Customer performs actions
acc1.deposit(500)
acc1.withdraw(300)

# 3. System checks final state
acc1.show_balance()
```

→ Attributes hold the data securely.

→ Methods ensure safe interactions.

Why Developers Use OOP

Without Classes

Variables float globally. Managing thousands of independent account balances leads to messy logic and high error rates.

With OOP

Every customer is contained within an object. It provides clean structure, high reusability, and a vastly scalable system.

Real Engineering

This isn't just theory. Bundling data with the actions that modify that data is exactly how real banking software is architected.

2

Independent Memory Spaces

```
acc1 = BankAccount("Amit", 100
acc2 = BankAccount("Rahul", 500
acc1.deposit(500)

acc1.show_balance() # 1500
acc2.show_balance() # 500
```

Why are the balances different? Because objects don't share data. `acc1` and `acc2` exist in entirely separate

Practice Task 1 — ATM Machine

Build the System

Design a class called `ATM` that handles machine cash levels.

- → **Attribute:** `balance` (How much cash the machine holds).
- → **Method:** `deposit()` (Admin adds cash).
- → **Method:** `withdraw()` (User takes cash, machine loses cash).
- → **Method:** `check_balance()` (Admin checks levels).

More Practice Systems

Student System

Class: `Student`

Attributes: `name`, `marks`
Methods: `show_result()` to print Pass/Fail depending on marks.

Wallet System

Class: `Wallet`

Attributes: `owner`, `money`
Methods: `add_money()`, `spend_money()`.

Mobile Phone

Class: `Phone`

Attributes: `brand`, `battery`
Methods: `charge()`, `use_phone()` (drains battery).

Day 4 Summary

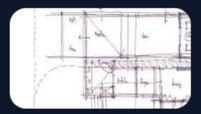
Class: The Real-world blueprint.
Object: The Real entity instance.
Memory: Multiple objects are strictly independent.
Methods: Functions that simulate real actions.

The OOP Shift: You no longer separate data and functions. They are bundled together. You are now designing real systems — not just writing code!

Image Sources

https://mojostumer.com/wp-content/uploads/bb-plugin/cache/6-20-landscape-8c29ef622b3fb60c90ebb07b11dd1068-5fad554f344ec.jpg

Source: mojostumer.com



https://www.lifeofanarchitect.com/wp-content/uploads/2019/08/Architectural-Sketch-Series-Schematic-Design-04.jpg

Source: www.lifeofanarchitect.com

https://static.vecteezy.com/system/resources/previews/057/645/517/non_2x/online-financial-transaction-icon-digital-payment-exchange-between-computer-and-laptop-screens-symbol-of-internet-banking-fintech-and-global-finance-secure-money-transfer-concept-vector.jpg

Source: www.vecteezy.com

https://thefintechtimes.com/wp-content/uploads/2021/02/iStock-1000736200-e1615910115248.jpg

Source: thefintechtimes.com

https://img.freepik.com/premium-vector/programmer-working-desk-with-multiple-monitors-displaying-code-dark-mode-technology-setup-software-development-vector-illustration_126712-42282.jpg

Source: ru.freepik.com